

Universal Template Parameters

Document #: P1985R2
Date: 2020-05-15
Project: Programming Language C++
Audience: Evolution Working Group Incubator
Reply-to: Mateusz Pusz ([Epm Systems](mailto:mateusz.pusz@gmail.com))
<mateusz.pusz@gmail.com>
Gašper Ažman
<gasper.azman@gmail.com>
Bengt Gustafsson
<bengt.gustafsson@beamways.com>
Colin MacLean
<ColinMacLean@lbl.gov>

Contents

1	Introduction	2
2	Change Log	2
2.1	R1 -> R2	2
2.2	R0 -> R1	2
3	Motivation	2
3.1	apply	2
3.2	fwd	3
4	Proposed Solution	3
4.1	“Easy” version	3
4.2	“Mathematically Correct” version	3
4.3	Mechanism	4
4.3.1	Specializing class templates on parameter kind	4
4.3.2	Eager or late checking?	4
4.3.3	Template <i>template-parameter</i> binding	5
4.4	Clarifying examples	5
4.4.1	Single parameter examples	5
4.4.2	Pack Expansion Example	6
4.4.3	Example of parsing ambiguity (if late check)	6
5	Example Applications	7
5.1	Enabling higher order metafunctions	7
5.2	Making dependent <code>static_assert(false)</code> work	7
5.3	Checking whether a type is a specialization of a given template	7
5.4	Universal alias as a library	8
5.5	Impacts on the specialization of class templates	9
5.6	Impact on the partial specialization of NTTP templates	10
5.7	Impacts on the specialization of variable templates	11
5.8	Integration with reflection	12
5.9	Library fundamentals II TS detection idiom	12

6	Covariance and Contravariance	12
6.1	As of C++20	12
6.2	Design space exploration for <code>template auto</code>	14
7	Other Considered Syntaxes	15
7.1	. and ... instead of <code>template auto</code> and <code>template auto ...</code>	15
8	Compendium: Open Design Questions	15
8.1	Eager or Late Checking	15
8.2	Easy or Correct	15
9	Further work (for context and planning): Cohesive template parameter theory	16
10	Acknowledgements	16
11	References	17

1 Introduction

We propose a universal template parameter. This would allow for a generic `apply` and other higher-order template metafunctions, as well as certain type traits.

2 Change Log

2.1 R1 -> R2

Having found overwhelming support for the feature in EWGI, and a concern about contra- and co-variance of the `template auto` parameters, we include discussion of these topics in the paper, together with the decision and comparison tables that underpin the decision.

The decision was to explore both and put it to a vote.

Added list of questions for EWGI.

2.2 R0 -> R1

- Greatly expanded the number of examples based on feedback from the BSI panel.
- Clarified that we are proposing eager checking

3 Motivation

3.1 `apply`

Imagine trying to write the `apply` metafunction.

It works in C++20 if one only considers type template parameters:

```
template <template <typename...> typename F, typename... Args>
using apply = F<Args...>;

template <typename X>
```

```
struct G { using type = X; };

static_assert(std::is_same<apply<G, int>, G<int>>{}); // OK
```

As soon as `G` tries to take any kind of NTTP (non-type template-parameter) or a template *template-parameter*, `apply` becomes impossible to write; we need to provide analogous parameter kinds for every possible combination of parameters:

```
template <template <typename> typename F>
using H = F<int>;
apply<H, G> // error, can't pass H as arg1 of apply, and G as arg2
```

While `apply` is very simple, a metafunction like `bind` or `curry` runs into the same problems.

3.2 fwd

A simpler example we should also use is `fwd`, which merely forwards its second template argument to its first, without the attempt of being variadic:

```
template <template <typename> typename F, typename arg>
using fwd = F<arg>;
```

4 Proposed Solution

Introduce a way to specify a truly universal template parameter that can bind to anything usable as a template argument.

There are two ways we can go about specifying this, depending on whether we want to be mathematically correct, or less powerful, imprecise, but “easy” to use. (See the co/contravariance discussion below).

4.1 “Easy” version

We spell it `template auto`. The syntax is somewhat up for debate in this version of the solution.

```
template <template <template auto...> typename F, template auto... Args>
using apply = F<Args...>;

apply<G, int>; // OK, G<int>
apply<H, G>;  // OK, G<int>
```

4.2 “Mathematically Correct” version

We need to introduce both a *anything* parameter (which we’ll spell `template auto` as above), and a *wildcard* parameter (which, given the work in pattern matching, we are probably free to spell `__`).

We can then define `apply` as follows:

```
template <template <__...> typename F, template auto... Args>
using apply = F<Args...>;
```

`fwd` is similar:

```
template <template <__> typename F, temmplate auto arg>
using fwd = F<arg>;
```

In this mechanism, `__` really is an “unconstrained” pattern match, and does not declare a parameter kind - it’s illegal to declare a `template <__ x> struct foo {};`, for instance.

4.3 Mechanism

4.3.1 Specializing class templates on parameter kind

The new universal template parameter introduces similar generalizations as the `auto` universal NTTP did; in order to make it possible to pattern-match on the parameter, class templates need to be able to be specialized on the kind of parameter as well:

```
template <template auto>
struct X;

template <typename T>
struct X<T> {
    // T is a type
    using type = T;
};

template <auto val>
struct X<val> : std::integral_constant<decltype(val), val> {
    // val is an NTTP
};

template <template <typename> F>
struct X<F> {
    // F is a unary metafunction
    template <typename T>
    using func = F<T>;
};
```

This alone allows building enough traits to connect the new feature to the rest of the language with library facilities, and rounds out template parameters as just another form of a compile-time parameter.

4.3.2 Eager or late checking?

There exists a choice in whether we check the usage of a universal template parameter eagerly or upon instantiation. Consider:

```
template <template auto Arg>
struct Y {
    using type = Arg; // can only alias a type
    // Q: late (on Y<1>) or early (on parse) error?
};
```

We could either:

- Eagerly check this and hard-error on parsing the template.
- Late-check and hard-error only if `Y` is *instantiated* with anything but a type.

4.3.2.1 What eager checking looks like

This paper **leans towards eager checking**, thus allowing the utterance of a universal template parameter only in:

- *template-parameter-list* as the declaration of such a parameter
- *template-argument-list* as the usage of such a parameter. It must bind exactly to a declared `template auto` template parameter.

This avoids any parsing issues by being conservative. We can always extend the grammar to allow usage in more contexts later.

Examples that inform the reasoning:

```
template <template auto Arg>
void f() {
    using type = Arg<int>; // error, not valid for types and values
    auto value = Arg.foo(); // error, not valid for types and class templates
    auto x = Arg::member; // error, not valid for class templates and values
};
```

We avoid ambiguity by allowing the use of the universal template parameter solely in ways that are valid for any kind of thing (value, type, or class template) it may represent. It just so happens that the only such use is passing it as an argument to a template and wait for something else to pattern-match it out.

4.3.2.2 What late checking looks like

If we chose to go with late checking, we'd be doing what we currently do with dependent identifiers - they are treated as values, and require “typename” and “template” to disambiguate.

The primary use-case of passing these identifiers as template arguments remains OK, but and treating them like this also eases specification, and is more consistent with the way the language currently works. This is the more “adventurous” option, however.

4.3.3 Template *template-parameter* binding

We have to consider the two phases of class template (and template alias) usage: parsing, and substitution.

Let's take `apply` again:

```
template <template <_...> typename F, template auto... Args>
using apply = F<Args...>;
```

This *parses*, because `F` is *declared* as a class template parameter taking a pack of `template auto`, while `Args` is being expanded into a place that takes `template auto` parameters.

When we pass a metafunction such as `is_same` as `F`:

```
using result = apply<std::is_same, int, 1>; // error
```

everything is known, so the compiler is free to check dependent expansions at instantiation time.

4.4 Clarifying examples

4.4.1 Single parameter examples

```

template <int> struct takes_int {};
template <typename T> using takes_type = T;
template <template auto> struct takes_anything {};
template <template <typename> typename F> struct takes_metafunc {};

template <template <template auto> typename F, template auto Arg>
struct fwd {
    using type = F<Arg>; // ok, passed to template auto parameter
}; // ok, correct definition

void f() {
    fwd<takes_int, 1>{}; // ok; type = takes_int<1>
    fwd<takes_int, int>{}; // error, takes_int<int> invalid
    fwd<takes_type, int>{}; // ok; type = takes_type<int>
    fwd<takes_anything, int>{}; // ok; type = takes_anything<int>
    fwd<takes_anything, 1>{}; // ok; type = takes_anything<1>
    fwd<takes_metafunc, takes_int>; // ok; type = takes_metafunc<takes_int>
    fwd<takes_metafunc, takes_metafunc>{}; // error (1)
}

```

(1): `takes_metafunc` is not a metafunction on a *type*, so `takes_metafunc<takes_metafunc>` is invalid (true as of C++98).

4.4.2 Pack Expansion Example

It is interesting to think about what happens if one expands a non-homogeneous pack of these. The result should not be surprising:

```

template <template auto X, template auto Y>
struct is_same : std::false_type {};
template <template auto V>
struct is_same<V, V> : std::true_type {};

template <template auto V, template auto ... Args>
struct count : std::integral_constant<
    size_t,
    (is_same<V, Args>::value + ...) > {};

// ok, ints = 2:
constexpr size_t ints = count<int, 1, 2, int, is_same, int>::value;

```

4.4.3 Example of parsing ambiguity (if late check)

The reason for eager checking is that not doing that could introduce parsing ambiguities. Example courtesy of [P0945R0], and adapted:

```

template <template auto A>
struct X {
    void f() { A * a; }
};

```

The issue is that we do not know how to parse `f()`, since `A * a;` is either a declaration or a multiplication.

If we treated `A` as just a dependent name, this always parses as a multiplication.

Original example from [P0945R0]:

```
template <typename T> struct X {
    using A = T::something; // P0945R0 proposed universal alias
    void f() { A * a; }
};
```

5 Example Applications

This feature is very much needed in very many places. This section lists examples of usage.

5.1 Enabling higher order metafunctions

This was the introductory example. Please refer to the [Proposed Solution](#).

Further example: curry:

```
template <typename <template auto...> typename F,
        template auto ... Args1>
struct curry {
    template <template auto... Args2>
    using func = F<Args1..., Args2...>;
};
```

5.2 Making dependent `static_assert(false)` work

Dependent static assert idea is described in [P1936R0] and [P1830R1]. In the former the author writes:

Another parallel paper [P1830R1] that tries to solve this problem on the library level is submitted. Unfortunately, **it cannot fulfill all use-case since it is hard to impossible to support all combinations of template template-parameters in the dependent scope.**

The above papers are rendered superfluous with the introduction of this feature. Observe:

```
// stdlib
template <bool value, template auto Args...>
inline constexpr bool dependent_bool = value;
template <template auto... Args>
inline constexpr bool dependent_false = dependent_bool<false, Args...>;

// user code
template <template <typename> typename Arg>
struct my_struct {
    // no type template parameter available to make a dependent context
    static_assert(dependent_false<Arg>, "forbidden specialization.");
};
```

5.3 Checking whether a type is a specialization of a given template

Main discussion of feature in [P2098R0] by Walter Brown and Bob Steagall.

When writing template libraries, it is useful to check whether a given type is a specialization of a given template. When our templates mix types and NTTPs, this trait is currently impossible to write in general. However, with the universal template parameter, we can write a concept for that easily as follows.

```
// is_specialization_of
template <typename T, template <template auto...> typename Type>
inline constexpr bool is_specialization_impl = false;

template <template auto... Params, template <template auto...> typename Type>
inline constexpr bool is_specialization_impl<Type<Params...>, Type> = true;

template <typename T, template <template auto...> typename Type>
concept is_specialization_of = is_specialization_impl<T, Type>;
```

With the above we are able to easily constrain various utilities taking class templates:

```
template <auto N, auto D>
struct ratio {
    static constexpr decltype(N) n = N;
    static constexpr decltype(D) d = D;
};

template <is_specialization_of<ratio> R1, is_specialization_of<ratio> R2>
using ratio_mul = simplify<ratio<
    R1::n * R2::n,
    R1::d * R2::d
>>;
```

or create named concepts for them:

```
template <typename T>
concept is_ratio = is_specialization_of<ratio>;

template <is_ratio R1, is_ratio R2>
using ratio_mul = simplify<ratio<
    R1::n * R2::n,
    R1::d * R2::d
>>;
```

This concept can then be easily used everywhere:

```
template <is_specialization_of<std::vector> V>
void f(V& v) {
    // valid for any vector
}
```

5.4 Universal alias as a library

While this paper does not try to relitigate Richard Smith's [P0945R0], it does provide a solution to aliasing anything as a library facility, without running into the problem that [P0945R0] ran into.

Observe:

```
template <template auto Arg>
struct alias /* undefined on purpose */;

template <typename T>
```



```

struct alias<T> { using value = T; }

template <auto V>
struct alias<V> : std::integral_constant<decltype(V), V> {};

template <template <template auto...> typename Arg>
struct alias<Arg> {
    template <template auto... Args> using value = Arg<Args...>;
};

```

We can then use alias when we *know* what it holds:

```

template <template auto Arg>
struct Z {
    using type = alias<Arg>;
    // existing rules work
    using value = typename type::value; // dependent
}; // ok, parses

Z<int> z1; // ok, decltype(z1)::value = int
Z<1> z; // error, alias<1>::value is not a type

```

5.5 Impacts on the specialization of class templates

Universal template arguments enable what appears like overloading of class templates by specializing a unimplemented primary template taking a pack of universal template parameters.

```

template <template auto...> struct my_container;

template <typename T> struct my_container<T> {
    my_container(T* data, size_t count);
    // A basic implementation
};

template <typename T, typename A> struct my_container<T, A> {
    my_container(T* data, size_t count);
    my_container(T* data, size_t count, const A& alloc);
    // An implementation using an allocator A
};

template <typename T, size_t SZ> struct my_container<T, SZ> {
    my_container(T* data, size_t count);
    // An implementation with an internal storage of SZ bytes
};

template <typename T> my_container(T*, size_t) -> my_container<T>;
template <typename T, typename A> my_container(T*, size_t, const A&) -> my_container<T, A>;

```

As the primary template is not defined there is only a default constructor implicit deduction guide. The explicit deduction guides select the first specialization unless an allocator object is given in which case the second specialization is selected. To select the third specialization the template arguments must be explicitly given.

5.6 Impact on the partial specialization of NTTP templates

It is a common pattern in C++ to use SFINAE to constrain template parameter types:

```
template <typename T, typename=void>
struct A;

template <template <typename> typename T, typename U>
struct A<T<U>,std::enable_if_t<std::is_integral_v<U>>>
{
    // Implementation for templates with integral parameter types
};

template <template <typename> typename T, typename U>
struct A<T<U>,std::enable_if_t<std::is_floating_point_v<U>>>
{
    // Implementation for templates with floating point parameter types
};

template <typename T>
struct X {};

A<X<int>> a; // Uses integral partial specialization
```

The covariant behavior of `template auto` allows a similar pattern to be used for NTTPs.

```
template <typename T, typename=void>
struct B;

template <template <template auto> typename T, auto U>
struct B<T<U>,std::enable_if_t<std::is_integral_v<decltype(U)>>>
{
    // Implementation for templates with integral parameter types
};

template <template <typename> typename T, auto U>
struct B<T<U>,std::enable_if_t<std::is_floating_point_v<decltype(U)>>>
{
    // Implementation for templates with floating point parameter types
};

template <int I>
struct Y {};

B<Y<5>> b; // Uses integral partial specialization
```

This pattern cannot currently be used with NTTPs as `auto` behaves contravariantly:

```
template <typename T, typename=void>
struct C;

template <template <auto> typename T, auto U>
struct C<T<U>,std::enable_if_t<std::is_integral_v<decltype(U)>>>
{};

template <int I>
```

```

struct Z1 {};

template <auto I>
struct Z2 {};

C<Z1<5>> c1; // Error: T does not match Z1
C<Z2<5>> c2; // Ok: NTPP can only be `auto`

```

With partial specialization, covariant behavior is desired. We rely on SFINAE to check that `T` accepts `U` and wish to accept any type of template which can take an integral NTPP in this example. `template auto` allows such a pattern to work without modifying the behavior of `auto`.

5.7 Impacts on the specialization of variable templates

Universal template parameters can be used to implement what appears like overloading of variable templates.

The problem of not being able to delete the base case then becomes more pressing than currently as selecting the base case should trigger an error, but will be selected when none of the specializations matches. This is solved by [P2041R0], which is assumed in the example below.

```

// Metafunction to find a tuple element by a type predicate.
template <template <typename> typename Pred, size_t Pos, typename Tuple>
constexpr size_t tuple_find() {
    if constexpr (Pos == tuple_size<Tuple>())
        return npos;
    else if constexpr (Pred<remove_cvref_t<tuple_element_t<Pos, Tuple>>>::value)
        return Pos;
    else
        return tuple_find<Pred, Pos + 1, Tuple>();
}

template <template <typename> typename Pred, typename Tuple>
constexpr size_t tuple_find() { return tuple_find<Pred, 0, Tuple>(); }

// Helper to bind the first arguments of a provided template
template <template <template auto...> TPL, template auto... Bs> struct curry {
    template <template auto... Ts> using func = TPL<Bs..., Ts>;
};

template <template auto... Ps>
constexpr auto tuple_find_v = delete;

template <template <typename> typename Pred, typename Tuple>
constexpr size_t tuple_find_v<Pred, Tuple> = tuple_find<Pred, Tuple>();

template <template <typename> typename Pred, size_t Pos, typename Tuple>
constexpr size_t tuple_find_v<Pred, Tuple> = tuple_find<Pred, Pos, Tuple>();

// Convenience specialization for use with binary predicate
template <template <typename, typename> typename Pred, typename M, typename Tuple>
constexpr size_t tuple_find_v<Pred, M, Tuple> = tuple_find_v<curry<Pred, M>::template func, Tuple>;

// Convenience specialization to match particular type.
template <typename T, typename Tuple>

```

```
constexpr size_t tuple_find_v<T, Tuple> = tuple_find_v<std::is_same, T, Tuple>;
```

This example is a metafunction to find a matching element in a tuple based only on its type. Unfortunately in C++20 `tuple_find` can only be implemented as a constexpr function if we want it to be usable with or without the start position (which mimics the `std::find` function in the value domain).

With universal template parameters we can specialize a template variable `tuple_find_v` to regain the symmetry with current tuple oriented metafunctions such as `tuple_size_v` and `tuple_element_t`.

Given a further overloaded constexpr function `tuple_size` we could simplify this to:

```
template <template auto... Ps> size_t tuple_find_v = tuple_find<Ps...>();
```

This relies on the power of universal template parameters in another way, and has the same simplicity as the current variable templates and type aliases of the standard library type traits.

To take the consistency a step further `tuple_find` could be implemented as a class template with universal template parameters as shown in the previous example instead of as the function it must be in C++20.

5.8 Integration with reflection

The authors expect that code using reflection will have a need for this facility. We should not reach for code-generation when templates will do, and so being able to pattern-match on the result of the reflection expression might be a very simple way of going from the consteval + injection world back to template matching.

The examples in this section are pending discussion with reflection authors.

Importantly, the authors do not see how one could write `is_specialization_of` with the current reflection facilities, because one would have no way to pattern-match. This is, however, also pending discussion with the authors of reflection.

5.9 Library fundamentals II TS detection idiom

TODO. Concievably simplifies the implementation and makes it far more general.

6 Covariance and Contravariance

6.1 As of C++20

Consider the two concepts:

```
template <typename T> concept A = true;
// B subsumes A
template <typename T> concept B = A<T> && true;
```

Covariance	Contravariance
<pre> auto returns_a() -> A; auto returns_b() -> B; auto f() { // OK, requirement less constrained than returns_b A x = returns_b(); // Error, requirement stricter than returns_b B y = returns_a(); } </pre>	<pre> template <template typename f> using puts_b = void; template <template <A> typename f> using puts_a = void; template using takes_b = void; template <A> using takes_a = void; using x = puts_b<takes_a>; // ok // Error, constraint mismatch (gcc) // clang accepts (in error) using w = puts_a<takes_b>; </pre>

We are used to the covariant case, but the usage of contravariant cases is not as common. The issue with `puts_a<takes_b>` is that `puts_a` requires a metafunction with a *wider interface* than one that just accepts Bs.

We have seen that concept-constrained template parameters behave correctly - covariantly on returns, contravariantly on parameters; but do other template parts as well?

Let's just replace A with `auto` and B with `int`:

```

template <template <int> typename f> using puts_int = void;
template <template <auto> typename f> using puts_auto = void;
template <int> using takes_int = void;
template <auto> using takes_auto = void;
using x = puts_int<takes_auto>; // OK
using w = puts_auto<takes_int>; // Error, but MSVC, GCC and clang all accept

```

TODO: Ask James Touton where in the standard this is made an error.

Function pointers do not convert in either co or contravariant ways:

```

struct X {}; struct Y : X {};
using f_of_x = void(*)(X&);
using f_of_y = void(*)(Y&);
// Error, no conversions between function pointers
f_of_y fy = static_cast<f_of_x>(nullptr);

using f_to_x = X&(*)();
using f_to_y = Y&(*)();
// Error, no conversions between function pointers
f_to_x xf = static_cast<f_to_y>(nullptr);

```

It does work for virtual function covariant return types:

```

struct ZZ {};
struct Z : ZZ {
    virtual auto f() -> Z&;
    virtual void g(Z&);
};
struct W : Z {
    auto f() -> W& override; // OK, covariant return type
    // Error, no contravariant parameter types
    void g(ZZ&) override;
};

```

How about parameter packs?

```
template <template <typename> typename f> using puts_one = void;
template <template <typename...> typename f> using puts_var = void;
template <typename> using takes_one = void;
template <typename...> using takes_var = void;
using x = puts_one<takes_var>; // OK in 17 and 20, Error in 14
using w = puts_var<takes_one>; // OK, compiles
```

Turns out parameter packs behave both ways (but also covariantly) - not what one would expect given the concepts example above.

6.2 Design space exploration for `template auto`

We have inconsistent behavior across the language. Let's say we did the right thing and made `template auto` behave contravariantly, like concepts.

```
template <template <auto> typename f> using puts_value = void;
template <template <template auto> typename f> using puts_any = void;
template <auto> using takes_value = void;
template <template auto> using takes_any = void;
using x = puts_value<takes_any>; // OK
using w = puts_any<takes_value>; // Error because contravariant.
```

But then, how do we write `apply`? Let's do it for a single argument to avoid complications with a covariant ...:

```
template <template <template auto T> typename f, template auto arg>
using apply1 = f<arg>;
```

The above is correct, but *useless* - it requires `f`'s signature to be `template <template auto T>`. What we want to express is that `f` is any kind of unary template metafunction, so we can pass in something like `template <int x> using int_constant = std::integral_constant<int, x>;`.

As a thought experiment, let's call the covariant version of `template auto` (with the meaning "deduce this from the argument") `__`:

```
template <template <__ T> typename f, template auto arg>
using apply1 = f<arg>;
```

This is what we want to express (and check at instantiation time), but now the `args` constraint is spelled differently from `f`'s constraint, and that might be very, very difficult to teach.

It also requires us to reserve an additional combination of tokens. Contrast what happens if we just made `template auto` behave covariantly (the way `__` behaves above) if used in that position. What do we lose?

At first glance, we lose the ability to define the contravariant meaning of `template auto` - a template parameter that *can* bind to anything. But can we get that back?

Recall that concepts behave contravariantly. Consider this one:

```
namespace std {
    template <template auto arg>
    concept anything = true;
};
```

If we relaxed the rule that only type concepts could appear in the template parameter list, we could say this:

```
template <template <std::anything ARG> typename takes_anything,
        template auto arg>
using apply1 = takes_anything<arg>;
```

This relaxation is *unlikely to happen* (given the past oral arguments in EWG), but we can still constrain with **requires**, albeit that is a far inferior-looking strategy.

This would behave *contravariantly*! While **template auto ARG** means “deduce”, a metafunction taking a concept (which behaves contravariantly!) that *anything* satisfies *has* to be a metafunction taking **template auto** (or **std::anything**).

In light of this, the paper authors have come to the conclusion that trying to make **template auto** behave contravariantly is all downside and little upside. The example to follow with **template auto** should be the behavior of ... (deduction behavior, both co- and contravariant).

This allows for usage to look the same as declaration, and like to bind to like. We anticipate the feature being much easier to teach this way, and in the rare cases when someone really needs the contravariant behavior, they will know to use a library-provided concept. It is also consistent with the rest of the non-concept template language.

Of course, the library-concept solution requires an additional relaxation of constraints syntax, but at least it’s feasible in the future, as opposed to requiring a second difficult-to-teach token sequence.

7 Other Considered Syntaxes

In addition to the syntax presented in the paper, we have considered the following syntax options:

7.1 . and ... instead of **template auto** and **template auto ...**

```
template <template <...> typename F, . x, . y, . z>
using apply3 = F<x, y, z>;
```

The reason we discarded this one is that it is very terse for something that should not be commonly used, and as such uses up valuable real-estate.

8 Compendium: Open Design Questions

See discussions above to inform these choices; this section just compiles the design questions for EWG and EWGI.

8.1 Eager or Late Checking

Whether to:

- specify **eager** checking with conservative rules on use of identifiers, or;
- specify **late** checking on instantiation and use the C++20 rules for parsing dependent expressions for them.

8.2 Easy or Correct

Whether to:

- (**correct**) reserve *two* token sequences (**template auto** and **__**) and have a single defined meaning for both, or;
- (**easy**) reserve *one* token sequence (**template auto**) and switch its meaning in a context-dependent manner, losing the ability to mean **template auto** in a *template template-parameter*, since in that context, **template auto** would mean **__**.

9 Further work (for context and planning): Cohesive template parameter theory

This work is **not a part of the proposal**, but we mention it here because this paper falls into the wider framework of cleaning up template parameter theory.

This section is basically wholly thought up by James Touton. We thank him for his ideas.

The core of the problem is that `...` behaves in a context-dependent manner; the “easy” way, as outlined above.

We fundamentally have the following list of template parameter constraints (ignoring concepts - those merely further constrain these atomic kinds):

- atomic constraints (may introduce identifiers, read “accepts” before the bullet):
 - anything: `template auto`
 - type: `typename T`
 - value, type-unconstrained: `auto V`
 - value, type-constrained: `int I`
 - metafunction-of: `template <parameter-constraints> typename F`
 - any-number-of *constraint*: `constraint...`
- metafunction parameter constraints:
 - atomic constraints (accepts *atomic constraint*):

```
template <template <auto...> typename F> struct f {  
    // the 'auto' part: F accepts any value  
    F<1> x;           // instantiated with an int  
    F<true> y;        // and also a bool  
    // the '...' part: F handles any arity  
    F<true, 111> z;   // and also a bool and a 111  
    F<> w;            // and also without params  
    static_assert(!std::is_same_v<decltype(x), decltype(y)>);  
};
```

- constraint relaxation (means “no constraint in this direction”):
 - “no constraint on arity”: C++20 ...

```
template <template <int ...> typename F, int ... Args>  
using apply_ints = F<Args...>;  
apply_ints<f, 1>; // works  
template <int x> square : std::integral_constant<int, x*x> {};  
apply_ints<square, 2>; // also works
```

- “no constraint on kind”: covariant `template auto` or `__`
- “no constraint on type”: covariant `auto`, no proposed spelling yet

We need some way to disambiguate the `...` case to be correct.

10 Acknowledgements

Special thanks and recognition goes to [Epam Systems](#) for supporting Mateusz’s membership in the ISO C++ Committee and the production of this proposal.

Gasper would likewise like to thank his employer, Citadel Securities Europe, LLC, for supporting his attendance in committee meetings.

Colin MacLean would also like to thank his employer, Lawrence Berkeley National Laboratory, for supporting his ISO C++ Committee efforts.

A big thanks also to the members of the BSI C++ panel for their review and commentary.
Thanks also to James Touton for walking Gašper through the covariance/contravariance design space.

11 References

- [P0945R0] Richard Smith. 2018. Generalizing alias declarations.
<https://wg21.link/p0945r0>
- [P1830R1] Ruslan Arutyunyan. 2019. std::dependent_false.
<https://wg21.link/p1830r1>
- [P1936R0] Ruslan Arutyunyan. 2019. Dependent Static Assertion.
<https://wg21.link/p1936r0>
- [P2041R0] David Stone. 2020. Deleting variable templates.
<https://wg21.link/p2041r0>
- [P2098R0] Walter E Brown, Bob Steagall. 2020. Proposing std::is_specialization_of.
<https://wg21.link/p2098r0>